Orderly — Problems, Solutions & Alternatives

As of 2026-09-18

Every problem found and worked on across this project's build-out, grouped by area. "Fixed" means verified (build/tests/live check); "Not fixed" names exactly what's blocking and who has to act.

Branding & UI

Problem	Status	Solution applied	Alternatives
Logo had ~65% dead canvas padding, baked-in white background	Fixed	Cropped to content, converted to true transparency	Vector redraw from scratch; outsource to a designer
"Log out" button wrapped to two lines on mobile navbar	Fixed	Added `whitespace-nowrap shrink-0` to the shared Button base style	Hamburger menu; icon-only button below a breakpoint
App used almost no icons	Fixed	Added a small hand-rolled SVG icon set, wired into StatCard/CardTitle/EmptyState	Pull in a library (lucide-react, heroicons) instead of hand-rolled SVGs

Crashes

Problem	Status	Solution applied	Alternatives
Stray comma in `SHOP_TYPES` array created a hole; `.find()` hit `undefined`, crashing the entire app on every load	Fixed	Removed the stray comma, cleaned the malformed entry	None needed — pure syntax bug
Deleting/recreating a Clerk account left the previous account's `shop`/`botStatus`/`whatsappLinked` in memory (no full page reload)	Fixed	Added a reset effect keyed on Clerk user-id changes	Force a full page reload on every sign-out (simpler, worse UX)

Onboarding / Connect WhatsApp

Problem	Status	Solution applied	Alternatives
Connect step showed fake Meta business-verification + template-approval UI that never matched the real bot	Fixed	Rewrote to the real QR-link flow (number, QR, checklist, risk callout)	Could have kept the Meta-style UI if Meta developer access had come through instead
No visible QR in the app — owner had to read it off a terminal	Fixed	Bot mirrors its QR to `public/bot-qr.png`; page polls and displays it live	Embed the QR as a live API/base64 endpoint instead of file mirroring
WhatsApp number saved unnormalized in 3 separate places (+91, spaces, dashes) — silently broke the bot's exact-match Supabase lookup	Fixed	Digit-only stripping on all 3 input locations, plus bot-side normalization	A phone-input library (libphonenumber-js) for fuller validation/formatting
No safeguard against accidentally editing the live WhatsApp number	Fixed	`LockableInput` — starts locked once a value exists, explicit unlock required	A confirmation modal/typed-phrase gate instead of a lock icon

Data honesty (mock vs. real)

Problem	Status	Solution applied	Alternatives
Dashboard showed fabricated revenue/orders/sales chart as if real	Fixed	Orders start empty; `weekSales` is a real aggregate of actual orders	—
Chats was 100% mock conversations	Fixed	Real `conversations`/`messages` tables; bot logs every real message; app polls (~5s)	Supabase Realtime channels instead of polling — instant updates, more moving parts
Owner replies typed in the app never actually sent on WhatsApp	Fixed	Bot polls for undelivered owner messages and relays them via the session	Realtime + webhook-triggered send instead of a poll loop

Bot reliability & connectivity

Problem	Status	Solution applied	Alternatives
No diagnostic logging — failures were completely silent	Fixed	Added received/ready/auth-failure/reply logging throughout	—
Number-mismatch only discoverable mid-conversation with a real customer	Fixed	Startup check queries Supabase immediately and logs match/no-match	—
WhatsApp's newer "@lid" senders (not in your contacts) get no reply — send throws inside whatsapp-web.js	Not fixed — open upstream bug	Detected and logged cleanly instead of a raw stack trace; no send fix exists	Switch to Baileys (tried earlier — blocked by an IP-level ban on new device links); switch to Twilio (paid, proven working); official Meta Cloud API (blocked — no dev account); wait for an upstream whatsapp-web.js fix; have the customer save the shop as a contact first
Bot replied to WhatsApp Channel/Group broadcasts (@newsletter, @g.us) as if they were customers	Fixed	Filtered out before any lookup/reply is attempted	—
`notice` event handler assumed `payload.instructions` was always an array — crashed on other shapes	Fixed	Defensive handling for array / string / missing / object shapes	—
No crash recovery — a Puppeteer page crash needed a manual restart	Fixed	`supervisor.js` auto-restarts on crash, stops after 5 crashes/30s	pm2; a systemd service; Docker with a restart policy
"WhatsApp linked" was a one-time flag, never re-verified — could silently go stale	Fixed	Bot heartbeat (~45s) + live "Active / Stale / No heartbeat" indicator in the app	—
Running `npm run dev` twice (or Ctrl+Z instead of Ctrl+C) left duplicate processes + a stale Chrome `SingletonLock`, blocking every new launch	Fixed this session (processes killed, locks removed)	Manual cleanup; recurring risk	`supervisor.js` could add a PID-file lock to refuse a second launch outright

Security & secrets

Problem	Status	Solution applied	Alternatives
`bot/.env.example` had real committed secrets (Supabase service_role key, Twilio SID/token)	Fixed	Replaced with placeholders; deduped the file	Rotate the real keys in their dashboards (still recommended, not done here)
Deleting a Clerk account leaves an orphaned Supabase row forever	Not deployed	Wrote a Clerk-webhook → Supabase Edge Function (svix-verified); needs dashboard deploy + webhook config	Accept orphaned rows as harmless (RLS makes them permanently unreachable); a scheduled cleanup cron instead of a webhook

Performance & testing

Problem	Status	Solution applied	Alternatives
Frontend bundle was 1.04MB (Vite's own build warning)	Fixed	Route-level code splitting — down to 575KB, heavy routes (Dashboard's recharts) load on demand	`manualChunks` config instead of per-route `lazy()`; raise the warning limit (masks the problem, doesn't fix it)
Zero automated test coverage anywhere in the repo	Fixed	33 tests added (order state machine + bot intent detection), Vitest in both packages	Broader unit coverage; end-to-end tests (Playwright) for full user flows

Worker-invite feature (built this session)

Problem	Status	Solution applied	Alternatives
No multi-user/staff support at all — one Clerk account was hard-assumed to be the whole shop	Fixed	`shop_members` table, `can_access_shop`/`can_manage_shop` RLS helpers, role resolution (owner/manager/staff) rewired through every query	Clerk Organizations (native multi-tenancy) instead of a custom `shop_members` table — bigger rearchitecture, more built-in
No invite delivery mechanism	Fixed	Reuses the existing wa.me deep-link pattern (same as the counter QR); `/invite/:token` accept page embeds Clerk's own sign-in/up	Email invites via Clerk's built-in invitation system instead of WhatsApp
No role permissions defined ("auth level" was vague)	Fixed	Concrete staff vs. manager split; gated on Dashboard revenue, Bot payments, Settings payments, avatar upload	—
Pending-invite lookup needs to work before the invitee has any relationship to the shop	Accepted trade-off, not fully closed	Any signed-in user can list *pending* invites (name/mobile/shop only, never accepted ones) — no backend server exists here to broker it tighter	Route through a service-role Edge Function instead (same pattern as the Clerk-deletion one) for a fully closed policy
Not every sensitive action is gated yet (e.g. stock toggle reachable in UI for staff, though RLS rejects the write)	Not fully fixed	Highest-value gates done (revenue, payments, avatar); exhaustive per-button gating not completed given scope	—

Missing features (not built — explicitly deferred)

Problem	Status	Solution applied	Alternatives
No real order-creation pipeline — the bot only answers questions, never places an order	Not built	— too large to bundle in alongside everything else, and money-handling code deserves undivided attention	Bot parses an "order" intent and creates order rows directly; a customer-facing web form instead of chat-only ordering; manual owner entry only

● Current live blocker (today)

Problem	Status	Solution applied	Alternatives
`shops` table has zero rows — confirmed directly via the service-role key. Every save the app has ever made has been silently rejected.	Root cause found, not yet fixed	Diagnosed: Supabase's Clerk third-party auth integration was very likely never enabled in the dashboard, so `auth.jwt()->>'sub'` always resolves NULL and every RLS-protected write silently fails	No real alternative — this exact one-time setup step (Dashboard → Authentication → Sign In/Providers → Third Party Auth → Clerk) is required for the Clerk+Supabase-RLS architecture this app already uses. The only bigger alternative would be dropping Clerk for Supabase's own native Auth — not worth it at this stage.
Migrations 0005–0010 confirmed never applied (direct query: `column shops.bot_status does not exist`)	Not yet fixed	—	Run all 6, in order, in the Supabase SQL Editor

Next step, in order: enable the Clerk integration → run migrations 0005–0010 → go through onboarding again with DevTools open → confirm a `shops` row actually appears → restart the bot.